

Objective 11: To implement Convolutional Neural Network (CNN) on MNIST Dataset.

```
# Import necessary Deep Learning libraries

import tensorflow as tf

from tensorflow.keras import layers, models

from tensorflow.keras.datasets import mnist

import matplotlib.pyplot as plt


# 1. Load the MNIST dataset

(X_train, y_train), (X_test, y_test) = mnist.load_data()


# 2. Preprocess the data

# Normalize pixel values to be between 0 and 1
X_train, X_test = X_train / 255.0, X_test / 255.0


# Reshape the data to include the "color channel" dimension.

# Grayscale is 1 channel. RGB would be 3.

# New shape: (number_of_images, 28, 28, 1)
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)


# 3. Build the Convolutional Neural Network

model = models.Sequential([

    # First Convolutional Block

    # 32 filters, each 3x3 in size. input_shape matches our reshaped data.
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)), # Shrinks the image by half


    # Second Convolutional Block

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
```

```

# Flatten the 2D maps into a 1D vector
layers.Flatten(),

# Standard Dense ANN layers for final classification
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax') # 10 output nodes for digits 0-9
])

# 4. Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Print a summary of the architecture
model.summary()

# 5. Train the model
print("\nTraining the CNN...")

# We use a small number of epochs (passes through the data) to save time
history = model.fit(X_train, y_train, epochs=5, validation_split=0.1)

# 6. Evaluate the model on the unseen test data
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")

```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 64)	102,464
dense_1 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)

Training the CNN...

Epoch 1/5
1688/1688 ————— 52s 30ms/step - accuracy: 0.9522 - loss: 0.1570 - val_accuracy: 0.9845 - val_loss: 0.0561
Epoch 2/5
1688/1688 ————— 46s 27ms/step - accuracy: 0.9836 - loss: 0.0515 - val_accuracy: 0.9887 - val_loss: 0.0390
Epoch 3/5
1688/1688 ————— 45s 27ms/step - accuracy: 0.9890 - loss: 0.0350 - val_accuracy: 0.9895 - val_loss: 0.0370
Epoch 4/5
1688/1688 ————— 44s 26ms/step - accuracy: 0.9920 - loss: 0.0258 - val_accuracy: 0.9898 - val_loss: 0.0419
Epoch 5/5
1688/1688 ————— 83s 27ms/step - accuracy: 0.9938 - loss: 0.0190 - val_accuracy: 0.9900 - val_loss: 0.0398
313/313 - 2s - 8ms/step - accuracy: 0.9907 - loss: 0.0317

Test Accuracy: 99.07%

10. To implement an Artificial Neural Network (ANN) for Classification and Regression.

Import necessary libraries

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_moons
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.inspection import DecisionBoundaryDisplay
```

1. Generate a complex, non-linear dataset (interlocking moons)

```
X, y = make_moons(n_samples=500, noise=0.15, random_state=42)
```

2. Split the data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Initialize the Artificial Neural Network (Multi-Layer Perceptron)

hidden_layer_sizes=(10, 10) means TWO hidden layers with 10 neurons each.

max_iter is the number of epochs (passes over the training data).

```
ann_classifier = MLPClassifier(
```

```
    hidden_layer_sizes=(10, 10),
```

```
    activation='relu',    # The standard activation function
```

```
    solver='adam',       # A highly efficient optimization algorithm
```

```
    max_iter=1000,
```

```
    random_state=42
```

```
)
```

4. Train the model

```
print("Training the Artificial Neural Network...")
```

```
ann_classifier.fit(X_train, y_train)
```

5. Evaluate the model

```
y_pred = ann_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"\nModel Accuracy: {accuracy * 100:.2f}%\n")
```

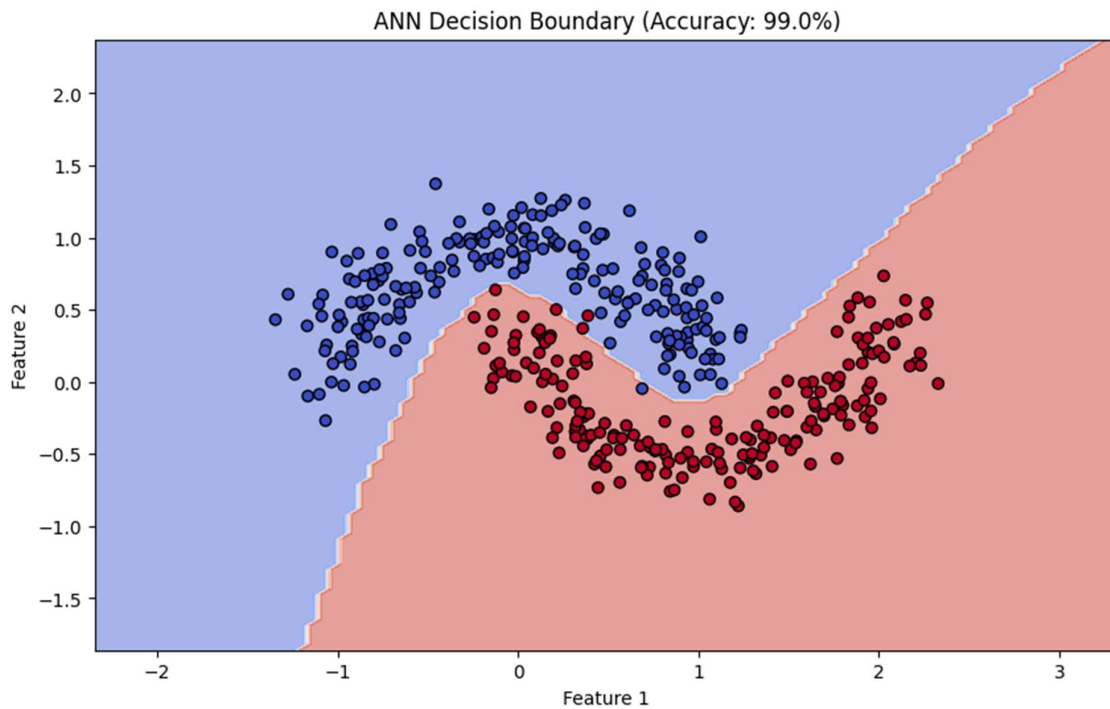
6. Visualize the complex Decision Boundary

```
plt.figure(figsize=(10, 6))
```

```
DecisionBoundaryDisplay.from_estimator(
    ann_classifier,
    X_train,
    response_method="predict",
    cmap=plt.cm.coolwarm,
    alpha=0.5,
    ax=plt.gca()
)
```

Scatter plot the actual training data

```
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=plt.cm.coolwarm, edgecolors='k')
plt.title(f"ANN Decision Boundary (Accuracy: {accuracy * 100:.1f}%)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```



Objective 9: To perform dimensionality reduction using Principal Component Analysis (PCA).

```
# Import necessary libraries
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load the Digits dataset (64 features per instance)
digits = datasets.load_digits()
X = digits.data
y = digits.target

# 2. Standardize the data (CRITICAL step for PCA)
# PCA is highly sensitive to the scale of the features.
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3. Initialize PCA

We tell it we only want to keep 2 principal components

```
pca = PCA(n_components=2)
```

4. Fit the model and transform the 64D data into 2D data

```
X_pca = pca.fit_transform(X_scaled)
```

5. Check how much information we actually kept

```
explained_variance = pca.explained_variance_ratio_
```

```
print(f"Variance captured by PC1: {explained_variance[0] * 100:.2f}%")
```

```
print(f"Variance captured by PC2: {explained_variance[1] * 100:.2f}%")
```

```
print(f"Total information retained: {sum(explained_variance) * 100:.2f}%\n")
```

6. Visualize the results

```
plt.figure(figsize=(10, 8))
```

We scatter plot the new 2D data, coloring them by their actual digit label (0-9)

```
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='tab10', alpha=0.7, s=15)
```

```
plt.colorbar(scatter, label='Actual Digit (0-9)')
```

```
plt.xlabel('Principal Component 1')
```

```
plt.ylabel('Principal Component 2')
```

```
plt.title('PCA: 64-Dimensional Digits Crushed to 2D')
```

```
plt.grid(True, linestyle='--', alpha=0.5)
```

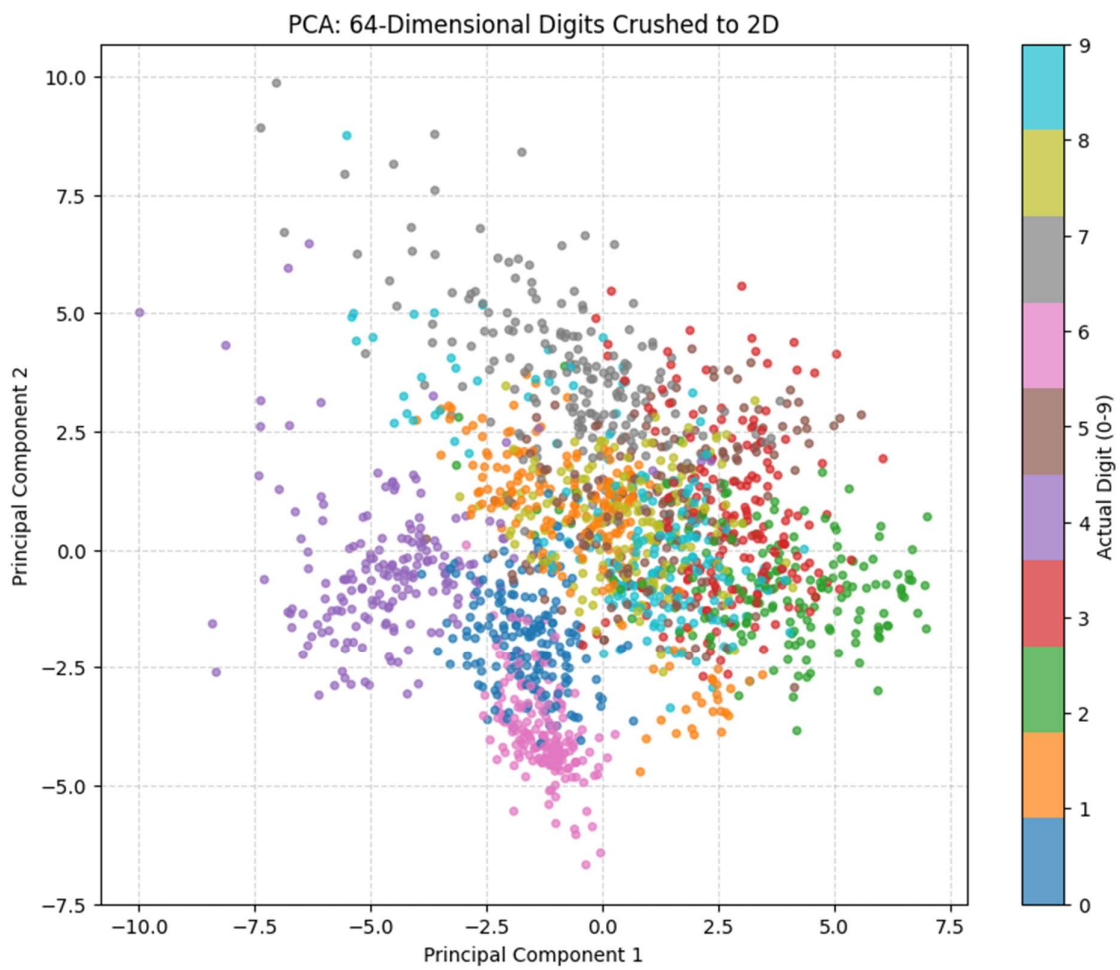
```
plt.show()
```

#Output:

Variance captured by PC1: 12.03%

Variance captured by PC2: 9.56%

Total information retained: 21.59%



Objective 8: To implement K-Means Clustering algorithm for unsupervised learning.

```
# Import necessary libraries
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_blobs
```

```
from sklearn.cluster import KMeans
```

```
# 1. Generate synthetic data for clustering
```

```
# We'll create 300 data points centered around 4 distinct blobs
```

```
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=0)
```

```
# 2. Initialize the K-Means algorithm
```

```
# We must explicitly tell the algorithm how many clusters to look for (K=4)
```

```
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=10, max_iter=300, random_state=0)
```

```
# 3. Fit the model and predict the cluster for each data point
```

```
# Notice there is no 'y_train' here because it is unsupervised!
```

```
y_kmeans = kmeans.fit_predict(X)
```

```
# 4. Extract the calculated cluster centers (centroids)
```

```
centroids = kmeans.cluster_centers_
```

```
print(f"Centroid Coordinates:\n{centroids}")
```

```
# 5. Visualize the results
```

```
plt.figure(figsize=(10, 6))
```

```
# Scatter plot the data points, coloring them by their assigned cluster
```

```
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis', alpha=0.7)
```

```
# Scatter plot the centroids as bold red X's
```

```
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, marker='X', edgecolor='black',  
label='Centroids')
```

```
plt.title("K-Means Clustering (K=4)")  
plt.xlabel("Feature 1")  
plt.ylabel("Feature 2")  
plt.legend()  
plt.grid(True, linestyle='--', alpha=0.5)  
plt.show()
```

#Output:

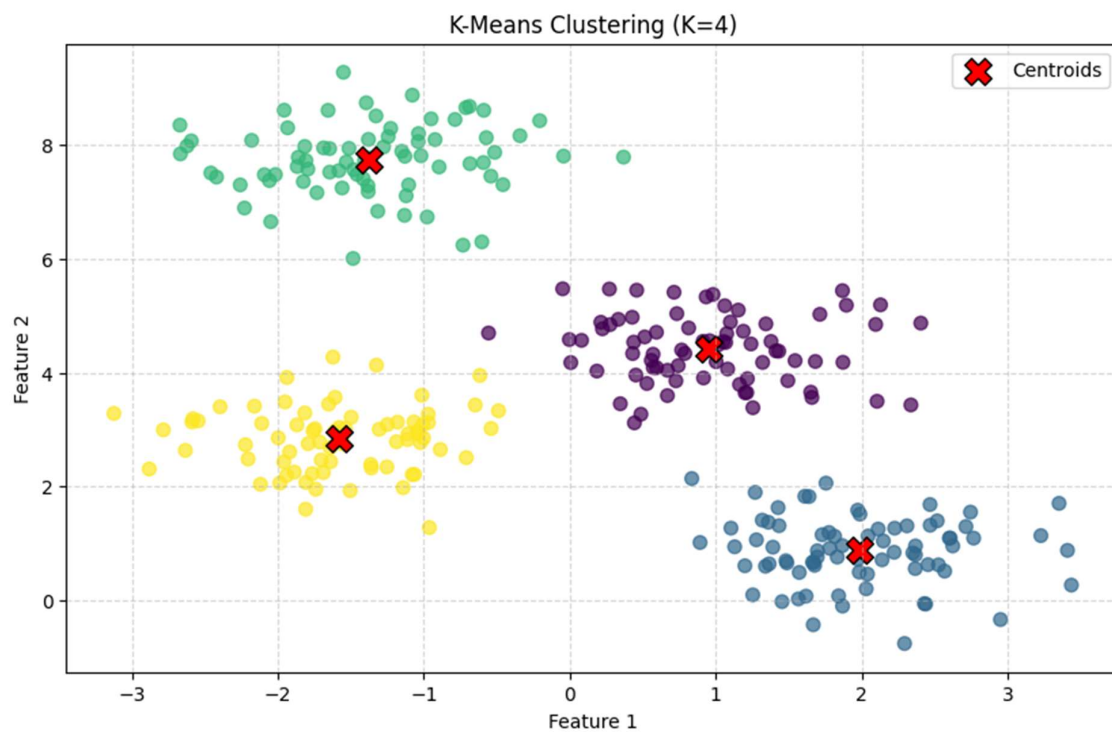
Centroid Coordinates:

```
[[ 0.94973532  4.41906906]
```

```
 [ 1.98258281  0.86771314]
```

```
[-1.37324398  7.75368871]
```

```
[-1.58438467  2.83081263]]
```



Objective 7: To implement Support Vector Machine (SVM) for classification tasks.

```
# Import necessary libraries

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report


# 1. Load the binary classification dataset

cancer_data = datasets.load_breast_cancer()

X = cancer_data.data # Features (tumor radius, texture, etc.)
y = cancer_data.target # Labels (0 = malignant, 1 = benign)


# 2. Split the data

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# 3. Initialize the Support Vector Classifier (SVC)

# We are using a linear kernel, which tries to separate data with a straight line/flat plane
svm_classifier = SVC(kernel='linear', C=1.0)


# 4. Train the model

print("Training the SVM model...")

svm_classifier.fit(X_train, y_train)


# 5. Make predictions

y_pred = svm_classifier.predict(X_test)


# 6. Evaluate the results

accuracy = accuracy_score(y_test, y_pred)

print(f"\nModel Accuracy: {accuracy * 100:.2f}%\n")
```

```
print("Classification Report:")  
print(classification_report(y_test, y_pred, target_names=cancer_data.target_names))
```

2nd Part

```
import matplotlib.pyplot as plt  
  
from sklearn import datasets  
  
from sklearn.model_selection import train_test_split  
  
from sklearn.svm import SVC  
  
from sklearn.inspection import DecisionBoundaryDisplay
```

1. Load the dataset, but ONLY take the first two features for 2D visualization

```
cancer_data = datasets.load_breast_cancer()  
  
X = cancer_data.data[:, :2] # Taking only 'mean radius' and 'mean texture'  
  
y = cancer_data.target
```

2. Split the data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Initialize and train the SVM

We use a linear kernel to draw a straight line

```
svm_classifier = SVC(kernel="linear", C=1.0)  
  
svm_classifier.fit(X_train, y_train)
```

4. Set up the plot

```
plt.figure(figsize=(10, 6))
```

Plot the decision boundary and the margins

```
DecisionBoundaryDisplay.from_estimator(  
    svm_classifier,  
    X_train,
```

```

    plot_method="contour",
    colors="k",
    levels=[-1, 0, 1], # -1 and 1 are the margins, 0 is the hyperplane
    alpha=0.5,
    linestyle=["--", "-", "--"],
    ax=plt.gca()
)

```

5. Scatter plot the actual training data points

```

scatter = plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=plt.cm.coolwarm, edgecolors='k')
plt.xlabel("Mean Radius")
plt.ylabel("Mean Texture")
plt.title("2D SVM Decision Boundary (Linear Kernel)")

```

6. Highlight the Support Vectors (the points that define the margin)

```

plt.scatter(
    svm_classifier.support_vectors[:, 0],
    svm_classifier.support_vectors[:, 1],
    s=100,
    linewidth=1.5,
    facecolors="none",
    edgecolors="k",
    label="Support Vectors"
)

```

```

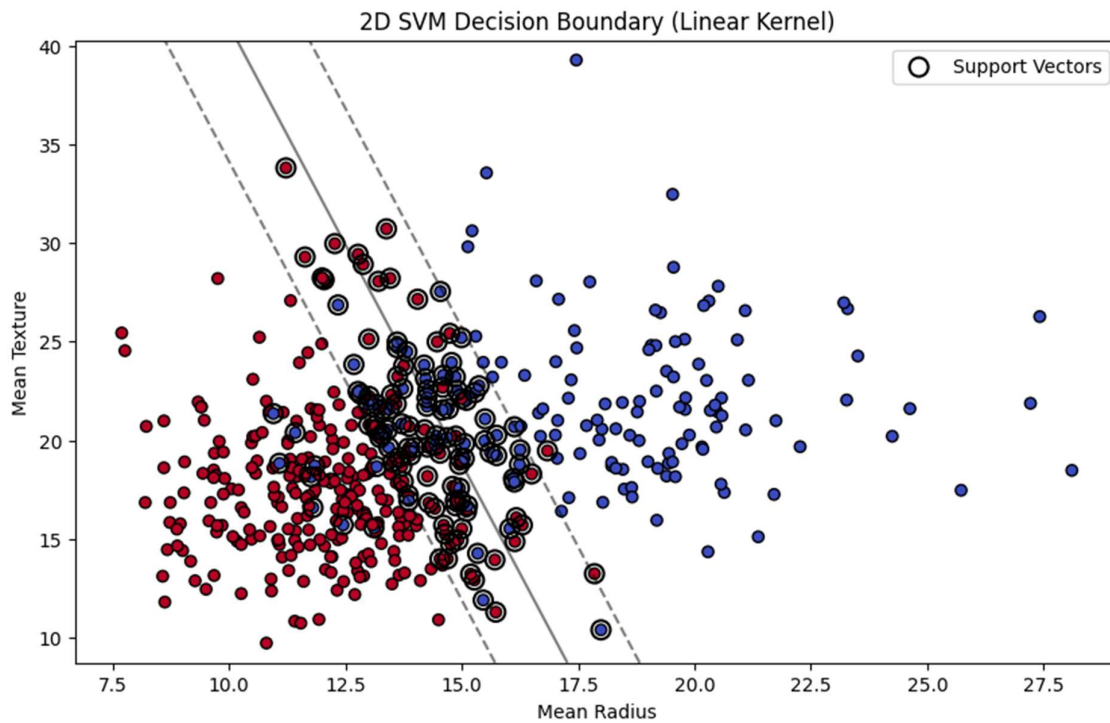
plt.legend()

```

```

plt.show()

```



Objective 6: To implement the Naïve Bayes classifier for supervised learning.

Import necessary libraries

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.metrics import accuracy_score
```

1. Create a highly simplified toy dataset of emails

'1' means Spam, '0' means Ham (Normal email)

emails = [

 "Win free money now", # Spam (1)

 "Claim your free prize", # Spam (1)

 "Urgent account update", # Spam (1)

 "Hey, see you at the meeting", # Ham (0)

 "Can we reschedule lunch?", # Ham (0)

 "Here is the project report" # Ham (0)

```
]
```

```
labels = [1, 1, 1, 0, 0, 0]
```

```
# 2. Convert the text into a matrix of token counts
```

```
# This creates a "vocabulary" and counts how often each word appears
```

```
vectorizer = CountVectorizer()
```

```
X_train = vectorizer.fit_transform(emails)
```

```
# 3. Initialize and train the Multinomial Naïve Bayes classifier
```

```
nb_classifier = MultinomialNB()
```

```
print("Training the text-based Naïve Bayes model...")
```

```
nb_classifier.fit(X_train, labels)
```

```
# 4. Test the model with brand new sentences it hasn't seen
```

```
test_emails = [
```

```
    "Get free money for the project", # Mix of spammy and normal words
```

```
    "Lunch meeting is urgent"        # Mix of normal and spammy words
```

```
]
```

```
X_test = vectorizer.transform(test_emails)
```

```
# 5. Make predictions and see the raw probabilities
```

```
predictions = nb_classifier.predict(X_test)
```

```
probabilities = nb_classifier.predict_proba(X_test)
```

```
# 6. Display the results
```

```
print("\n--- Classification Results ---")
```

```
for i, email in enumerate(test_emails):
```

```
    status = "SPAM" if predictions[i] == 1 else "HAM"
```

```
    spam_prob = probabilities[i][1] * 100
```

```
    print(f"Email: '{email}'")
```

```
    print(f"Prediction: {status} (Spam Probability: {spam_prob:.2f}%)\\n")
```

Objective 5: To implement the K-Nearest Neighbors (KNN) algorithm for classification.

```
from sklearn.datasets import make_moons

import matplotlib.pyplot as plt

import seaborn as sns

import pandas as pd


# Create synthetic 2D-data

x, y = make_moons(n_samples = 100, noise = 0.3, random_state = 42)


# Create a Dataframe for plotting

df = pd.DataFrame(x, columns = ["Feature1", "Feature2"])

df["Target"] = y


# Visualize the 2D data

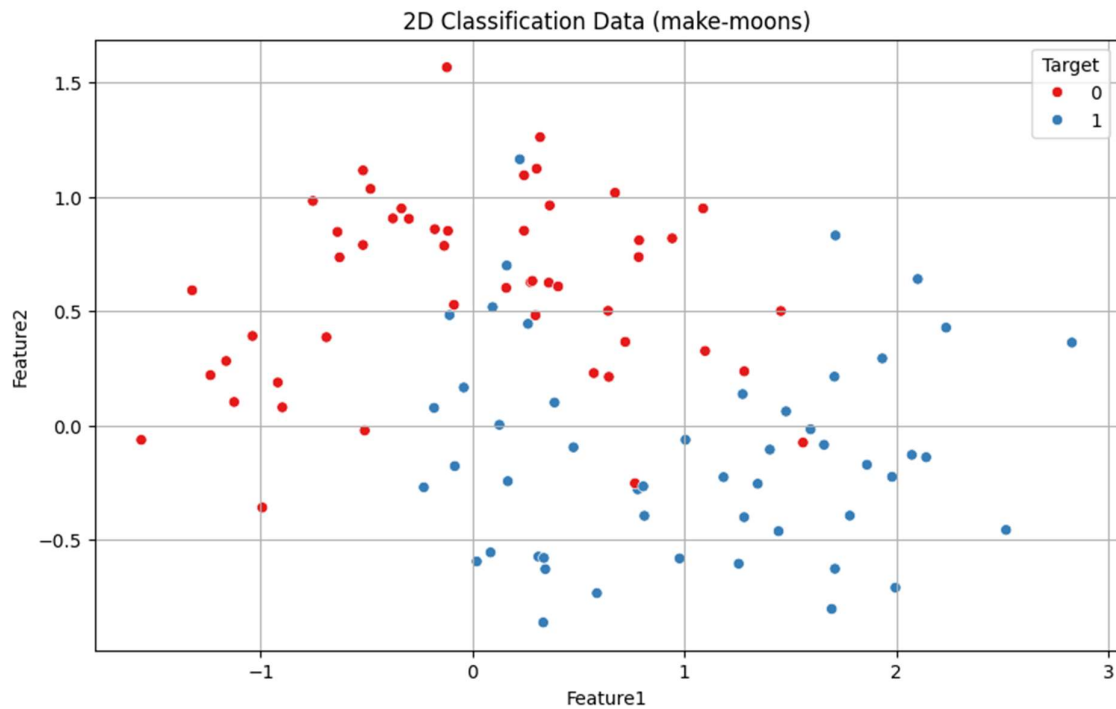
plt.figure(figsize = (10, 6))

sns.scatterplot(data= df, x="Feature1", y="Feature2", hue="Target", palette="Set1") # try : palette = "Set1 or viridis"

plt.title("2D Classification Data (make-moons)")

plt.grid(True)

plt.show()
```



```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Normalize the features
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(x)
```

```
# Split into train and test
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2, random_state = 42,  
stratify = y)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
# Train a KNN Classifier
```

```
knn = KNeighborsClassifier(n_neighbors = 5)
```

```
knn.fit(X_train, y_train)
```

```

# Predict & Evaluate

y_pred = knn.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Test Accuracy (k=5): {accuracy:.2f}")

from sklearn.metrics import confusion_matrix, classification_report, ConfusionMatrixDisplay

# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(confusion_matrix = cm, display_labels = ["Class0", "Class1"])

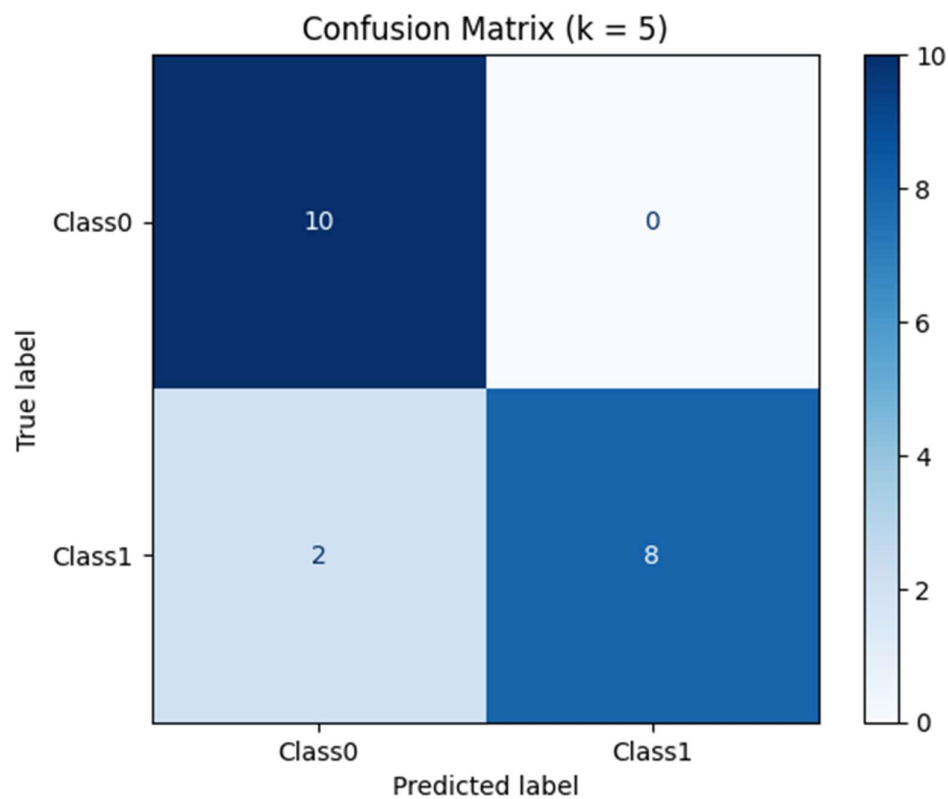
disp.plot(cmap = "Blues")

plt.title(f"Confusion Matrix (k = 5)")

plt.grid(False)

plt.show()

```



```

# Detailed Classification Report

print("Classification Report:")

```

```
print(classification_report(y_test, y_pred, target_names = ["Class0", "Class1"]))
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# Create mesh grid
```

```
x_min, x_max = X_scaled[:,0].min() - 1, X_scaled[:,0].max() + 1
```

```
y_min, y_max = X_scaled[:,1].min() - 1, X_scaled[:,1].max() + 1
```

```
xx, yy = np.meshgrid( # Renamed to xx, yy to avoid conflict
```

```
    np.arange(x_min, x_max, 0.01),
```

```
    np.arange(y_min, y_max, 0.01)
```

```
)
```

```
# Predict for every grid point
```

```
z_mesh = knn.predict(np.c_[xx.ravel(), yy.ravel()]) # Renamed prediction variable
```

```
# Reshape
```

```
z_mesh = z_mesh.reshape(xx.shape)
```

```
# Plot
```

```
plt.figure(figsize=(8,6))
```

```
plt.contourf(xx, yy, z_mesh, cmap=plt.cm.coolwarm, alpha=0.3)
```

```
sns.scatterplot(
```

```
    x=X_scaled[:,0],
```

```
    y=X_scaled[:,1],
```

```
    hue=df['Target'], # Using df['Target'] for the 1D labels
```

```
    palette='Set1',
```

```
    edgecolor='k'
```

)

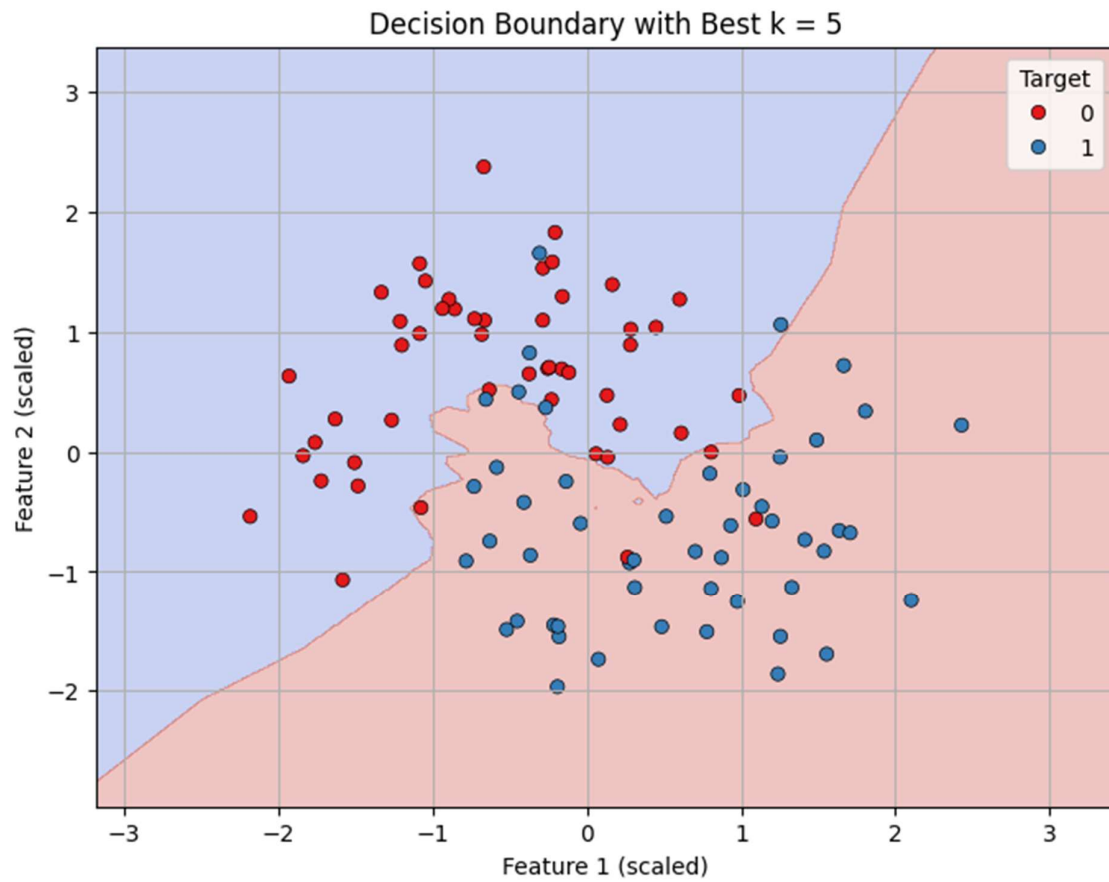
```
plt.title("Decision Boundary with Best k = 5")
```

```
plt.xlabel("Feature 1 (scaled)")
```

```
plt.ylabel("Feature 2 (scaled)")
```

```
plt.grid(True)
```

```
plt.show()
```



Objective 4: Logistic Regression

Step 5 : Scikit-Learn example (Iris Dataset)

```
import pandas as pd
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, precision_score, classification_report
```

```
# Load dataset
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
df = pd.DataFrame(X, columns=iris.feature_names)
```

```
df['Species'] = y
```

```
print(df.head())
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
# Model
```

```
model = LogisticRegression(max_iter=200)
```

```
# Train
```

```
model.fit(X_train, y_train)
```

```
# Predict

y_pred = model.predict(X_test)

# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred, average='weighted'))
print(classification_report(y_test, y_pred))
```

Ojective 3: Linear Regression

Step 1 : Import required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Step 2: Create Sample Dataset

data = {
    "Hours": [1,2,3,4,5,6,7,8,9,10],
    "Score": [15,20,30,40,45,55,65,70,80,90]
}
```

```
df = pd.DataFrame(data)
```

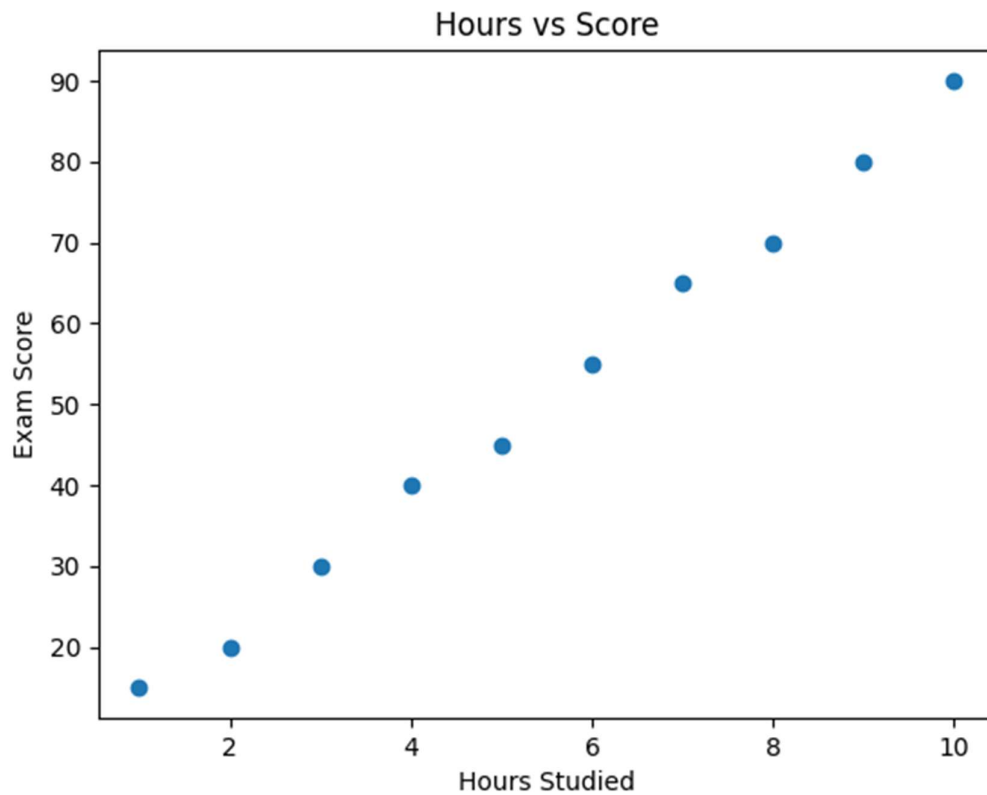
```
print(df)
```

Step 3: Visualize the Data

```
plt.scatter(df["Hours"], df["Score"])
plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
```

```
plt.title("Hours vs Score")
```

```
plt.show()
```



```
# Step 4: Prepare Features and Target
```

```
X = df[["Hours"]] # Independent variable
```

```
y = df["Score"] # Dependent variable
```

```
# Step 5: Split the Dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
```

```
)
```

```
print("Training data size:", len(X_train))
```

```
print("Testing data size:", len(X_test))
```

```
# Step 6: Train the Linear Regression Model
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)

# Step 7: Make Predictions
y_pred = model.predict(X_test)

print("Predicted values:", y_pred)
print("Actual values:", y_test.values)

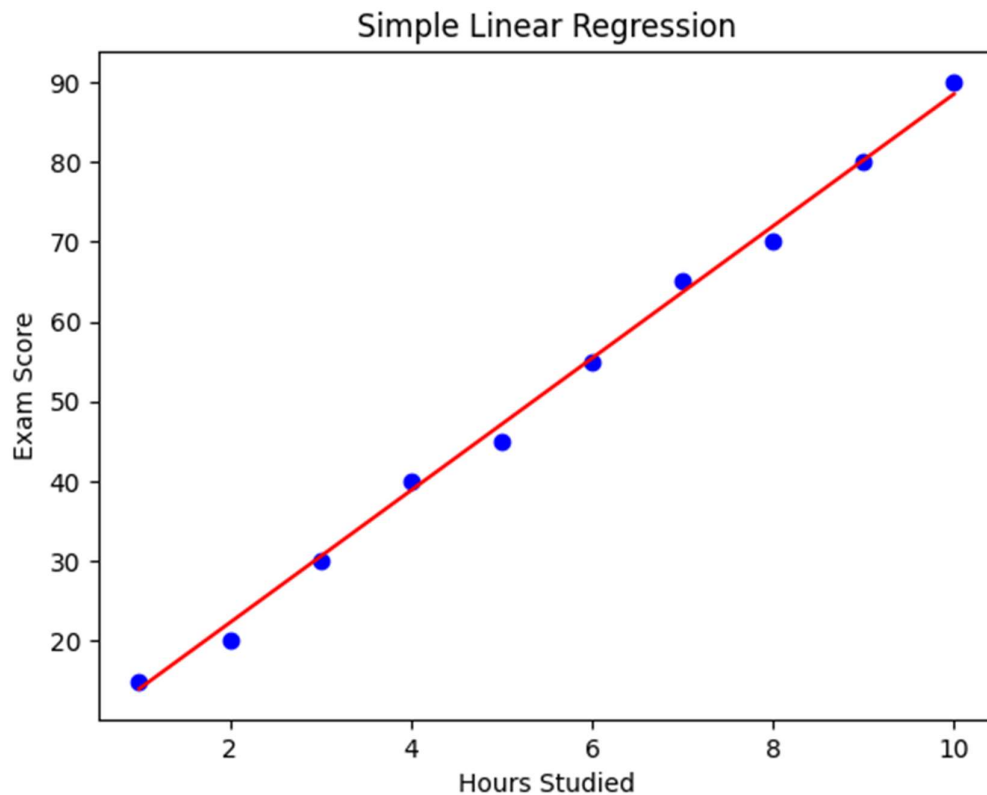
# Step 8: Evaluate Model Performance
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
print("R2 Score:", r2)

# Step 9: Plot Regression Line
plt.scatter(X, y, color="blue")
plt.plot(X, model.predict(X), color="red")

plt.xlabel("Hours Studied")
plt.ylabel("Exam Score")
plt.title("Simple Linear Regression")

plt.show()
```



Objective 2: Data Preprocessing

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, StandardScaler, MinMaxScaler
```

1. Create a "messy" raw dataset

```
data = {  
    'Country': ['France', 'Spain', 'Germany', 'Spain', np.nan, 'France'],  
    'Age': [44, 27, 30, 38, 40, np.nan],  
    'Salary': [72000, 48000, 54000, 61000, np.nan, 58000],  
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes']  
}
```

```
df = pd.DataFrame(data)
```

```
print("--- RAW DATA ---")
```

```
print(df, "\n")
```

2. Handle Missing Values (Imputation)

Replace missing numeric values with the column MEAN

```
num_imputer = SimpleImputer(strategy='mean')
```

```
df[['Age', 'Salary']] = num_imputer.fit_transform(df[['Age', 'Salary']])
```

Replace missing categorical values with the MOST FREQUENT value (mode)

```
cat_imputer = SimpleImputer(strategy='most_frequent')
```

```
df[['Country']] = cat_imputer.fit_transform(df[['Country']])
```

3. Encode Categorical Variables

Label Encoding for the Target Variable (Yes/No -> 1/0)

```
le = LabelEncoder()
```

```
df['Purchased'] = le.fit_transform(df['Purchased'])
```

One-Hot Encoding for the Features (Country -> multiple binary columns)

```
ohe = OneHotEncoder(sparse_output=False)
```

```
country_encoded = ohe.fit_transform(df[['Country']])
```

Get the new column names (Country_France, Country_Germany, etc.)

```
country_columns = ohe.get_feature_names_out(['Country'])
```

```
df_encoded = pd.DataFrame(country_encoded, columns=country_columns)
```

Drop the old 'Country' column and attach the new One-Hot columns

```
df = df.drop('Country', axis=1)
```

```
df = pd.concat([df_encoded, df], axis=1)
```

4. Feature Scaling & Normalization

Let's Standardize the Salary column (mean=0, std=1)

```
standard_scaler = StandardScaler()
```

```
df['Salary_Standardized'] = standard_scaler.fit_transform(df[['Salary']])
```

```
# Let's Normalize the Age column (squash between 0 and 1)
minmax_scaler = MinMaxScaler()
df['Age_Normalized'] = minmax_scaler.fit_transform(df[['Age']])

print("--- FULLY PREPROCESSED DATA ---")
# Drop the original unscaled columns to see the final result cleanly
df_final = df.drop(['Age', 'Salary'], axis=1)
print(df_final)
```

Objective 1: To study and implement basic python libraries

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error


# --- STEP 1: PANDAS (Data Creation & Structure) ---

# Simulating a dataset of Study Hours vs Exam Scores

raw_data = {

    'Study_Hours': [1.5, 2.0, 3.2, 4.0, 5.1, 5.5, 6.0, 6.8, 7.2, 8.0, 8.5, 9.2],

    'Exam_Score': [42, 48, 55, 60, 68, 72, 75, 83, 81, 89, 95, 98]

}


# Convert the dictionary into a highly functional Pandas DataFrame

df = pd.DataFrame(raw_data)


print("--- Pandas DataFrame Head ---")

print(df.head(4)) # View first 4 rows


print("\n--- Pandas Statistical Summary ---")

print(df.describe()) # Instant mean, std dev, min, max


# --- STEP 2: SCIKIT-LEARN (Data Splitting & Modeling) ---

# Isolate our feature (X) and target variable (y)

X = df[['Study_Hours']] # Features must be 2D array/DataFrame

y = df['Exam_Score'] # Target is a 1D Series


# Split into 80% training data and 20% test data
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train a Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model using Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)
print(f"\n--- Scikit-Learn Model Evaluation ---")
print(f"Mean Squared Error (MSE): {mse:.2f}")

# --- STEP 3: MATPLOTLIB (Data Visualization) ---
plt.figure(figsize=(9, 5))

# Plot the original data points
plt.scatter(df['Study_Hours'], df['Exam_Score'], color='royalblue', s=60, label='Actual Data')

# Generate a continuous line across the domain using our Scikit-learn model
X_line = np.linspace(df['Study_Hours'].min(), df['Study_Hours'].max(), 100).reshape(-1, 1)
y_line = model.predict(X_line)

# Plot the regression trend line
plt.plot(X_line, y_line, color='crimson', linewidth=2.5, label=f'Regression Line (MSE: {mse:.2f})')

# Customizing the chart elements
plt.title('Study Hours vs. Exam Performance', fontsize=14, fontweight='bold', pad=15)
plt.xlabel('Hours Studied', fontsize=11)
plt.ylabel('Exam Score (%)', fontsize=11)

```

```
plt.legend(loc='upper left')  
plt.grid(True, linestyle='--', alpha=0.6)  
  
# Display the final graph  
plt.show()
```